La Taberna del Bus de Eventos

Jesús Abraham Moreno Flores | Tarea de arquitectura de software

Problema. La taberna necesita coordinar pedidos, barra, cocina, música, sonido y notificaciones sin que cada componente invoque a los demás.

Solución. Los publicadores emiten objetos inmutables. EventBus distribuye cada objeto a los consumidores registrados para su clase exacta. Un consumidor puede reaccionar y publicar otro evento. Main crea y conecta las piezas.

Concurrencia. El bus reparte notificaciones de forma síncrona y breve. Barra y Cocina programan entregas independientes a los 2 y 5 segundos mediante ScheduledExecutorService. La banda anuncia canciones cada 4 segundos mientras las preparaciones siguen pendientes.

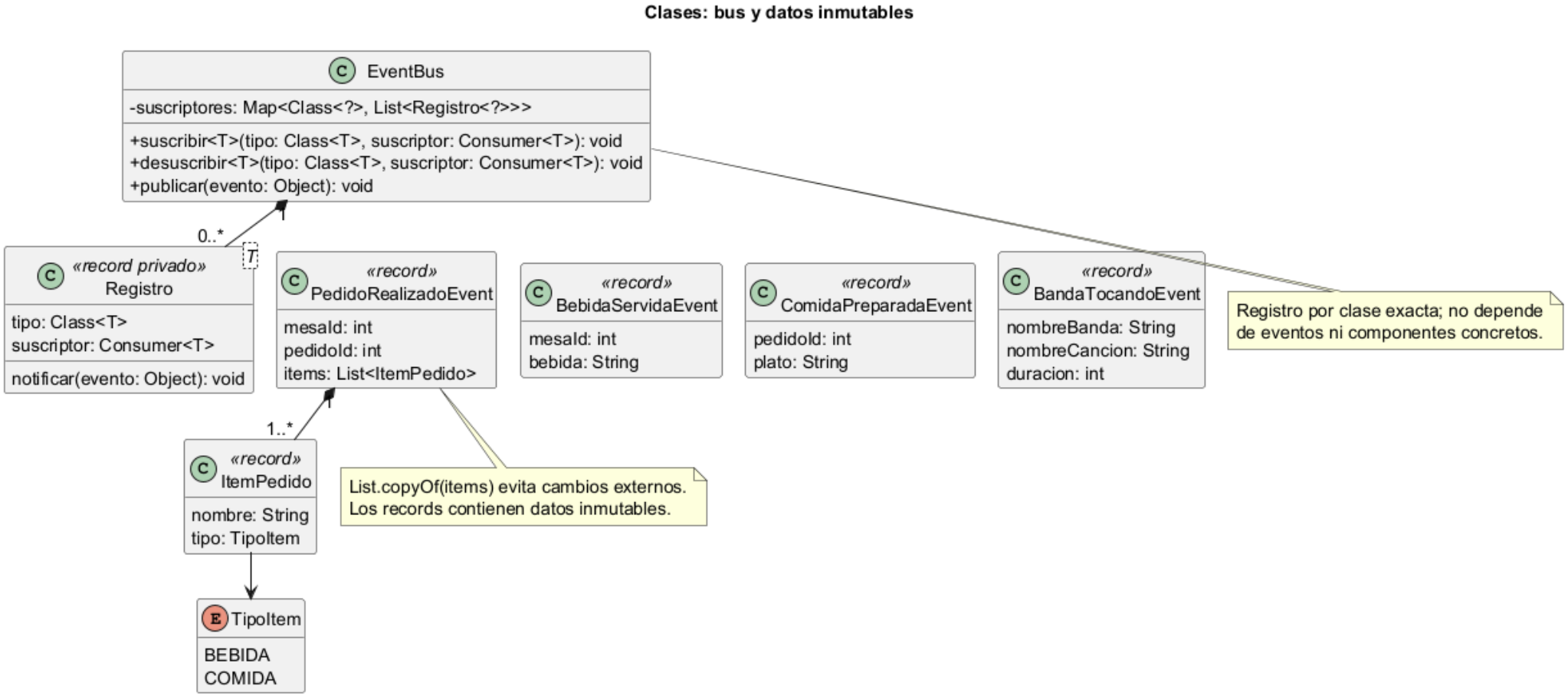
Suscripciones. Altas y bajas sincronizadas; el reparto usa una copia de los registros y ejecuta receptores fuera del bloqueo. Los cambios afectan publicaciones futuras. No se repite el registro del mismo Consumer. Una RuntimeException de un receptor se informa y no interrumpe al resto.

Datos. Los cuatro eventos son records. PedidoRealizadoEvent copia su lista de ItemPedido con List.copyOf; cada producto también es un record inmutable. Los datos se validan al construirlos.

Alcance. Consola, sin dependencias Java externas. El bus es local y no persiste mensajes ni incluye reintentos. La duración original de las canciones se conserva como dato; el intervalo musical se acelera para la demostración.

UML de clases: bus y eventos

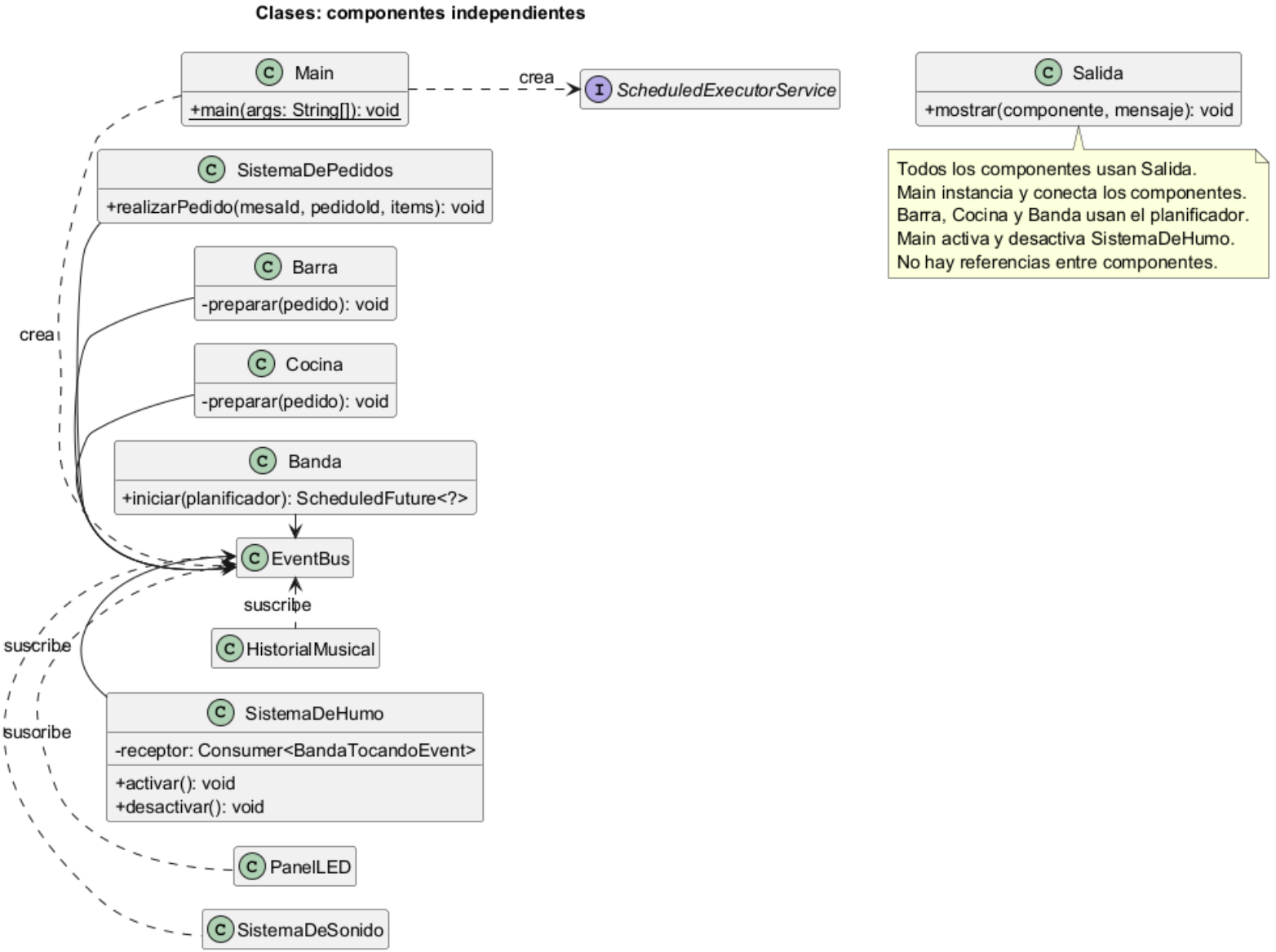
Arquitectura Event Bus | Java 21



La inmutabilidad incluye la lista del pedido y sus elementos.

UML de clases: componentes

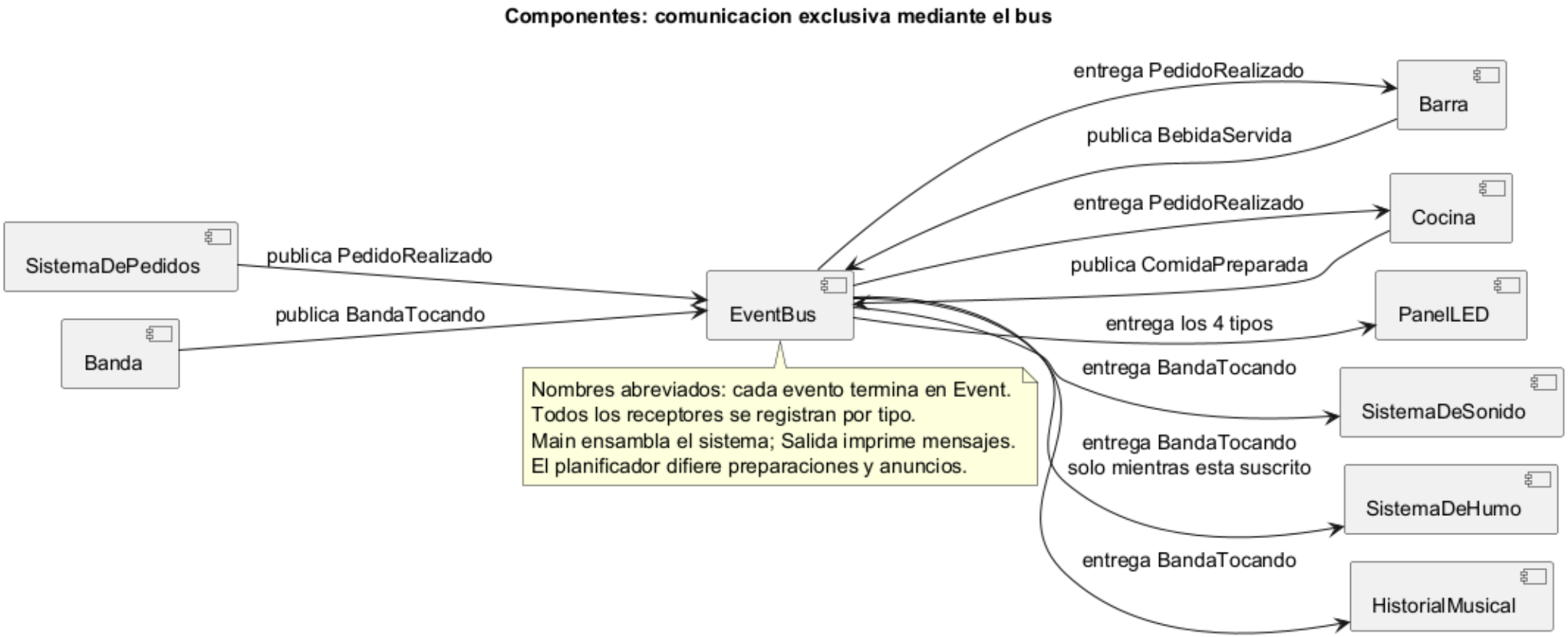
Arquitectura Event Bus | Java 21



Las flechas indican dependencias; los componentes colaboran a través del bus.

UML de componentes

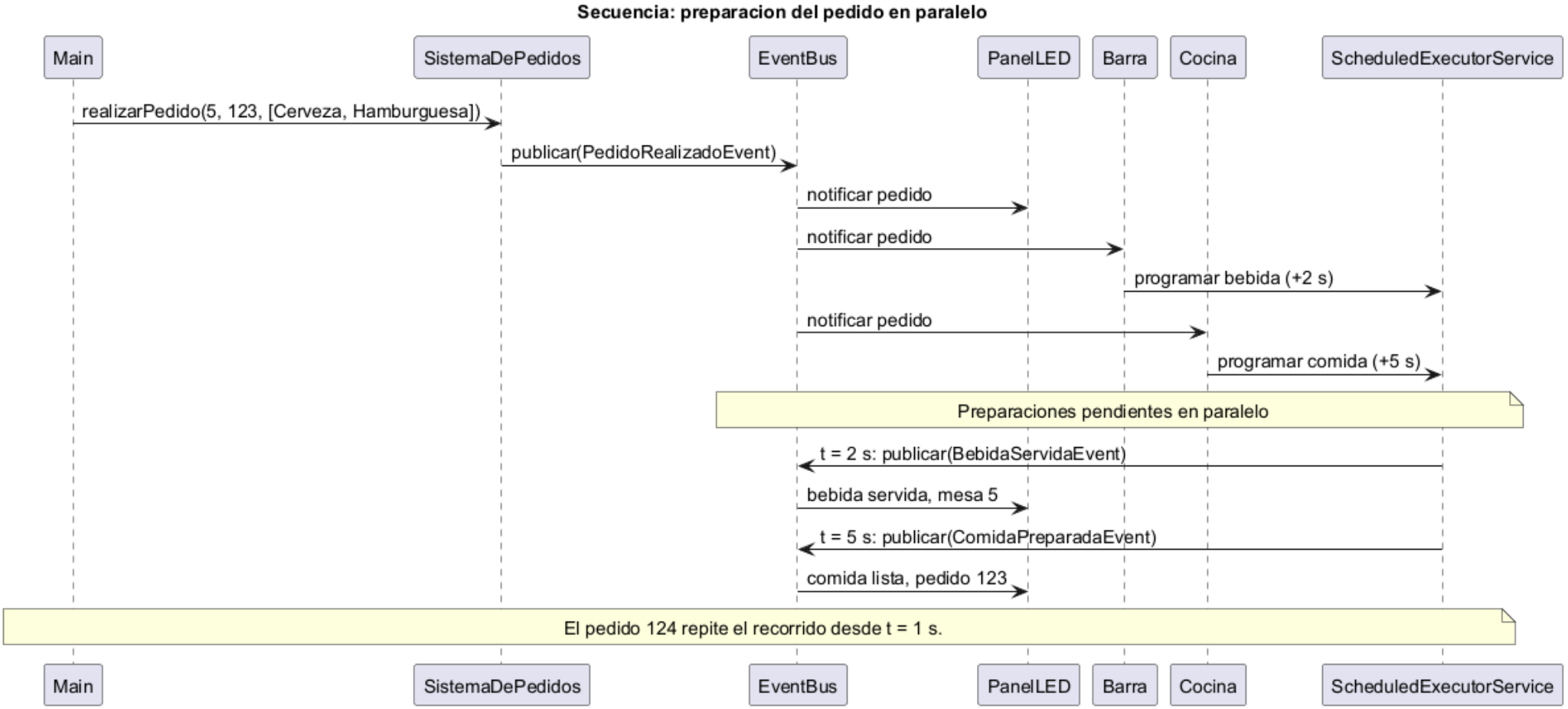
Arquitectura Event Bus | Java 21



Las flechas representan publicación y entrega; los receptores se registran por tipo.

UML de secuencia: pedidos

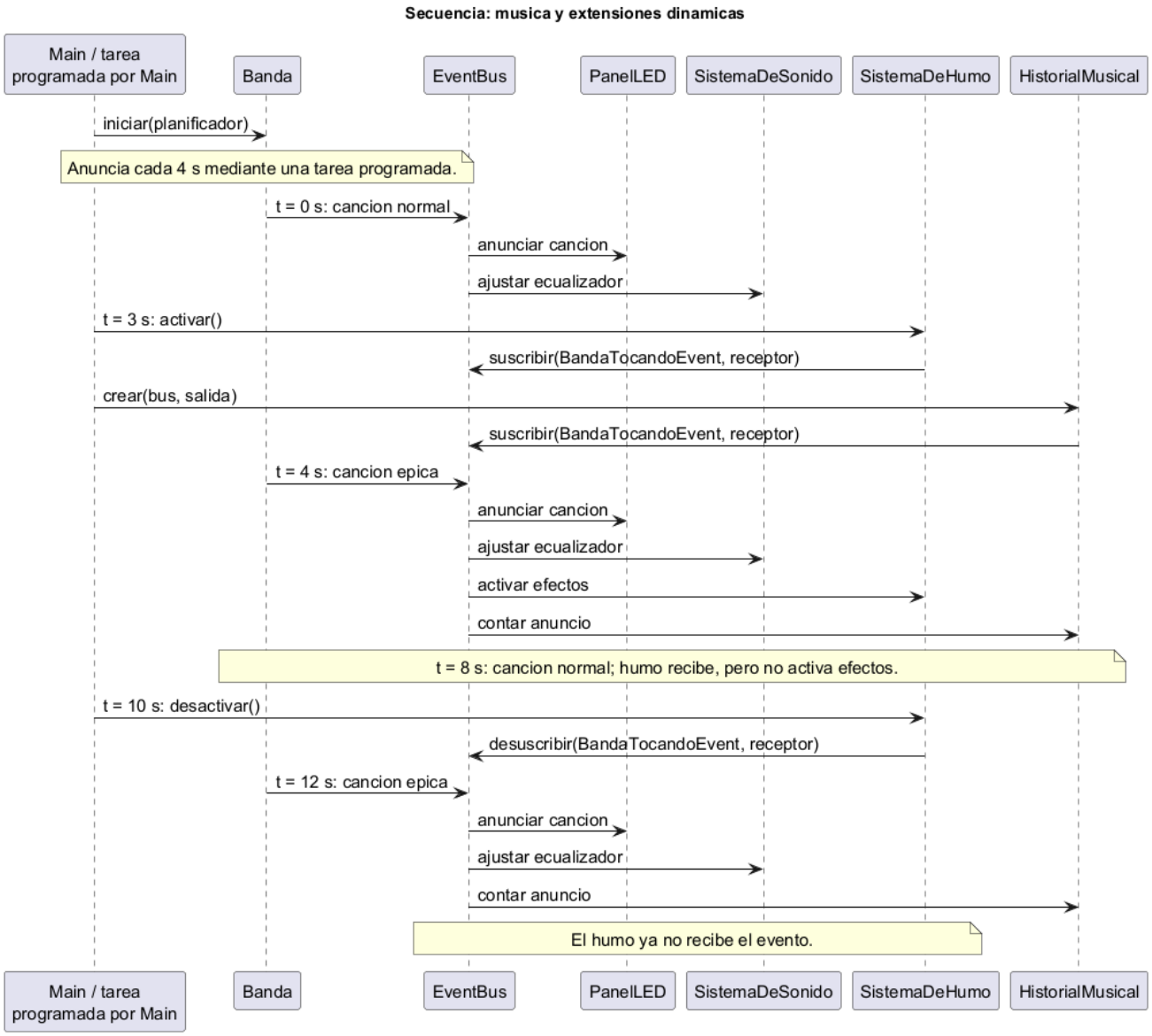
Arquitectura Event Bus | Java 21



Pedido 123; el pedido 124 sigue el mismo recorrido desde t = 1 s.

UML de secuencia: música

Arquitectura Event Bus | Java 21



Cada anuncio publica un BandaTocandoEvent; el humo se agrega y se retira en ejecución.

Escenario y comprobación

Arquitectura Event Bus | Java 21

0-3 s. Mesa 5 pide cerveza y hamburguesa (pedido 123). La banda anuncia El Algoritmo del Amor y sonido ajusta su ecualizador. A los 1 s, mesa 2 pide agua y pizza (pedido 124). El panel anuncia la cerveza a los 2 s y el agua a los 3 s. Main incorpora Humo e HistorialMusical a los 3 s.

4-6 s. Through the Fire and Flames activa el humo y registra el primer anuncio del historial. El panel anuncia la hamburguesa a los 5 s y la pizza a los 6 s. Las preparaciones se solapan con la música.

8-14 s. Se anuncia otra canción normal. A los 10 s se desuscribe el humo. A los 12 s vuelve la canción épica: panel, sonido e historial reaccionan; el humo ya no recibe el evento. A los 14 s termina la simulación y se cierra el planificador.

Abierto/cerrado. SistemaDeHumo e HistorialMusical se añaden en el segundo commit. Solo se cambia la composición en Main. Banda, EventBus y los componentes previos permanecen sin cambios. El humo conserva su Consumer para retirarlo correctamente.

Validación. Siete pruebas cubren registro/baja/tipos/duplicados; cambios durante el reparto y eventos anidados; fallos aislados; publicación concurrente; inmutabilidad; preparación en paralelo; y humo dinámico. docs/ejecucion.txt contiene una ejecución completa real con marcas de tiempo.

Ejecutar. JDK 21 o superior. En PowerShell: `./ejecutar.ps1`. Pruebas: `./ejecutar.ps1 -Pruebas`. Sin colores: `./ejecutar.ps1 -SinColor`. Los tiempos son aproximados; las líneas del mismo instante pueden intercalarse.

Fundamento y referencias

Arquitectura Event Bus | Java 21

Akamai describe productores, consumidores y un bus intermediario que dirige los eventos a receptores interesados. El proyecto aplica esa separación: quien anuncia un pedido o canción desconoce qué componentes reaccionan. Las preparaciones asíncronas se implementan con tareas programadas y un bus local sencillo.

Enunciado y rúbrica. La Taberna del Bus de Eventos, Instrucciones.pdf, y dos imágenes proporcionadas. README.md relaciona cada criterio con evidencia: bus, eventos inmutables, desacoplamiento, extensibilidad, UML y flujo funcional.

Akamai. What Is an Event Bus? https://www.akamai.com/glossary/what-is-an-event-bus. Consulta: 2 de noviembre de 2026.

Video indicado. Event Driven Architecture (EDA): Todo lo que DEBERÍAS saber | Tutorial en Español. https://www.youtube.com/watch?v=GdelFsukQwE. Se verificó el título, pero no se obtuvo una transcripción. No se le atribuyen explicaciones específicas.

PlantUML. Imágenes generadas con PlantUML 1.2025.0. El código se incluye a continuación; las fuentes .puml y PNG están en docs/diagramas. https://plantuml.com.

Repositorio. github.com/Jesus-Abraham-Moreno-Flores/EventBusPatternADS.

Código PlantUML

docs/diagramas/clases-eventos.puml

```
@startuml
!pragma layout smetana
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam defaultFontSize 14
hide empty members
title Clases: bus y datos inmutables
class EventBus {
  - suscriptores: Map<Class<?>, List<Registro<?>>>
  + suscribir<T>(tipo: Class<T>, suscriptor: Consumer<T>): void
  + desuscribir<T>(tipo: Class<T>, suscriptor: Consumer<T>): void
  + publicar(evento: Object): void
}
class "Registro<T>" as Registro <<record privado>> {
  tipo: Class<T>
  suscriptor: Consumer<T>
  notificar(evento: Object): void
}
class PedidoRealizadoEvent <<record>> {
  mesaId: int
  pedidoId: int
  items: List<ItemPedido>
}
class BebidaServidaEvent <<record>> {
  mesaId: int
  bebida: String
}
class ComidaPreparadaEvent <<record>> {
  pedidoId: int
  plato: String
}
class BandaTocandoEvent <<record>> {
  nombreBanda: String
  nombreCancion: String
  duracion: int
}
class ItemPedido <<record>> {
  nombre: String
  tipo: TipoItem
}
enum TipoItem {
  BEBIDA
  COMIDA
```

Código PlantUML (continuación)

docs/diagramas/clases-eventos.puml

```
}
EventBus "1" *-- "0..*" Registro
PedidoRealizadoEvent "1" *-- "1..*" ItemPedido
ItemPedido --> TipoItem
EventBus -[hidden]down- PedidoRealizadoEvent
PedidoRealizadoEvent -[hidden]right- BebidaServidaEvent
BebidaServidaEvent -[hidden]right- ComidaPreparadaEvent
ComidaPreparadaEvent -[hidden]right- BandaTocandoEvent
note bottom of EventBus
  Registro por clase exacta; no depende
  de eventos ni componentes concretos.
end note
note bottom of PedidoRealizadoEvent
  List.copyOf(items) evita cambios externos.
  Los records contienen datos inmutables.
end note
@enduml
```

Código PlantUML

docs/diagramas/clases-componentes.puml

```
@startuml
!pragma layout smetana
left to right direction
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam defaultFontSize 14
hide empty members
title Clases: componentes independientes
class Main {
  + {static} main(args: String[]): void
}
class EventBus
class SistemaDePedidos {
  + realizarPedido(mesaId, pedidoId, items): void
}
class Barra {
  - preparar(pedido): void
}
class Cocina {
  - preparar(pedido): void
}
class Banda {
  + iniciar(planificador): ScheduledFuture<?>
}
class SistemaDeSonido
class PanelLED
class SistemaDeHumo {
  - receptor: Consumer<BandaTocandoEvent>
  + activar(): void
  + desactivar(): void
}
class HistorialMusical
interface ScheduledExecutorService
class Salida {
  + mostrar(componente, mensaje): void
}
Main .right.> EventBus : crea
SistemaDePedidos -right-> EventBus
Barra -right-> EventBus
Cocina -right-> EventBus
Banda -right-> EventBus
SistemaDeSonido .left.> EventBus : suscribe
PanelLED .left.> EventBus : suscribe
```

Código PlantUML (continuación)

docs/diagramas/clases-componentes.puml

```
SistemaDeHumo -left-> EventBus
HistorialMusical .left.> EventBus : suscribe
Main .down.> ScheduledExecutorService : crea
ScheduledExecutorService -[hidden]down- Salida
note bottom of Salida
  Todos los componentes usan Salida.
  Main instancia y conecta los componentes.
  Barra, Cocina y Banda usan el planificador.
  Main activa y desactiva SistemaDeHumo.
  No hay referencias entre componentes.
end note
@enduml
```

Código PlantUML

docs/diagramas/componentes.puml

```
@startuml
!pragma layout smetana
left to right direction
skinparam shadowing false
skinparam defaultFontSize 14
title Componentes: comunicacion exclusiva mediante el bus
component SistemaDePedidos as Pedidos
component Barra
component Cocina
component Banda
component EventBus as Bus
component PanelLED as LED
component SistemaDeSonido as Sonido
component SistemaDeHumo as Humo
component HistorialMusical as Historial
Pedidos --> Bus : publica PedidoRealizado
Bus --> Barra : entrega PedidoRealizado
Barra --> Bus : publica BebidaServida
Bus --> Cocina : entrega PedidoRealizado
Cocina --> Bus : publica ComidaPreparada
Banda --> Bus : publica BandaTocando
Bus --> LED : entrega los 4 tipos
Bus --> Sonido : entrega BandaTocando
Bus --> Humo : entrega BandaTocando\nsolo mientras esta suscrito
Bus --> Historial : entrega BandaTocando
note bottom of Bus
  Nombres abreviados: cada evento termina en Event.
  Todos los receptores se registran por tipo.
  Main ensambla el sistema; Salida imprime mensajes.
  El planificador difiere preparaciones y anuncios.
end note
@enduml
```

Código PlantUML

docs/diagramas/secuencia.puml

```
@startuml
skinparam shadowing false
skinparam defaultFontSize 14
title Secuencia: preparacion del pedido en paralelo
participant Main
participant SistemaDePedidos as Pedidos
participant EventBus as Bus
participant PanelLED as LED
participant Barra
participant Cocina
participant "ScheduledExecutorService" as Reloj
Main -> Pedidos : realizarPedido(5, 123, [Cerveza, Hamburguesa])
Pedidos -> Bus : publicar(PedidoRealizadoEvent)
Bus -> LED : notificar pedido
Bus -> Barra : notificar pedido
Barra -> Reloj : programar bebida (+2 s)
Bus -> Cocina : notificar pedido
Cocina -> Reloj : programar comida (+5 s)
note over Bus, Reloj : Preparaciones pendientes en paralelo
Reloj -> Bus : t = 2 s: publicar(BebidaServidaEvent)
Bus -> LED : bebida servida, mesa 5
Reloj -> Bus : t = 5 s: publicar(ComidaPreparadaEvent)
Bus -> LED : comida lista, pedido 123
note over Main, Reloj : El pedido 124 repite el recorrido desde t = 1 s.
@enduml
```

Código PlantUML

docs/diagramas/secuencia-musica.puml

```
@startuml
skinparam shadowing false
skinparam defaultFontSize 15
title Secuencia: musica y extensiones dinamicas
participant "Main / tarea\nprogramada por Main" as Main
participant Banda
participant EventBus as Bus
participant PanelLED as LED
participant SistemaDeSonido as Sonido
participant SistemaDeHumo as Humo
participant HistorialMusical as Historial
Main -> Banda : iniciar(planificador)
note over Banda : Anuncia cada 4 s mediante una tarea programada.
Banda -> Bus : t = 0 s: cancion normal
Bus -> LED : anunciar cancion
Bus -> Sonido : ajustar ecualizador
Main -> Humo : t = 3 s: activar()
Humo -> Bus : suscribir(BandaTocandoEvent, receptor)
Main -> Historial : crear(bus, salida)
Historial -> Bus : suscribir(BandaTocandoEvent, receptor)
Banda -> Bus : t = 4 s: cancion epica
Bus -> LED : anunciar cancion
Bus -> Sonido : ajustar ecualizador
Bus -> Humo : activar efectos
Bus -> Historial : contar anuncio
note over Banda, Historial : t = 8 s: cancion normal; humo recibe, pero no activa efectos.
Main -> Humo : t = 10 s: desactivar()
Humo -> Bus : desuscribir(BandaTocandoEvent, receptor)
Banda -> Bus : t = 12 s: cancion epica
Bus -> LED : anunciar cancion
Bus -> Sonido : ajustar ecualizador
Bus -> Historial : contar anuncio
note over Bus, Humo : El humo ya no recibe el evento.
@enduml
```